

4. The `respond()` method in the target scene (which you must write) does whatever it needs to do to generate the data that was asked for, then returns the data to the scene manager.
5. The scene manager returns the data to the `request()` method in the `Scene` base class inherited by the current scene.
6. Finally, the `request()` method in the `Scene` base class returns the data to the original caller.

The same mechanism is used to implement `send()` and `sendAll()`. The only difference is that when sending a message to a scene or to all scenes, there is no data to be returned to the original caller.

Summary

In this chapter, I introduced two different ways to implement a program that incorporates multiple scenes. A state machine is a technique for representing and controlling the flow of execution through a series of states; you can use it to implement a program with a small number of scenes. The scene manager is designed to help you build larger multi-scene applications by providing navigation and a general way for scenes to communicate with each other. I also explained how the scene manager implements all this functionality.

The scene manager and the `Scene` base class provide clear examples of the three main tenets of object-oriented programming: encapsulation, polymorphism, and inheritance. Each scene is a good example of encapsulation because all the code and data of a scene is written as a class. Each scene class must be polymorphic, in that it must implement a common set of methods in order for it to work with calls from the scene manager. Finally, each scene inherits from a common `Scene` base class. Two-way communication between the scene manager and the `Scene` base class is implemented by every scene using the inherited methods and instance variables in the base class.